
Project 2 Report: C-Minus Syntax Analyzer

1. Introduction

The objective of this project was to implement a syntax analyzer (parser) for the C-Minus language, following the grammar defined in the course guide. The parser was built using the **Flex** tool (for lexical analysis) and **Bison** (for syntax analysis).

The program's main function is to read a C-Minus source code file, verify that its structure adheres to the grammar, and, as its output, generate a textual representation of the corresponding **Abstract Syntax Tree (AST)**.

2. Environment and Compilation Method

2.1. Development Environment

The project was developed and compiled in a Linux environment, using a Docker container (cs-compiler-hw:1.0). The key tools used were:

- **C Compiler:** gcc (GNU Compiler Collection)
- **Parser Generator:** bison (invoked as yacc)
- **Scanner Generator:** flex
- **Build System:** make

2.2. Compilation Process

The project was compiled using the provided Makefile. The process to generate the final executable, `cminus_parser`, consists of two commands:

1. **make clean:** (Optional) Cleans up object files (.o) and generated files (y.tab.c, lex.yy.c, etc.) from previous compilations.
2. **make cminus_parser:** Compiles all source files (.c, .l, .y) and links them to create the executable.

```
root@d5b5bed19d89:/work# make cminus_parser
yacc -d -v cminus.y
gcc -W -Wall -c main.c
gcc -W -Wall -c util.c
flex cminus.l
gcc -W -Wall -c lex.yy.c
lex.yy.c:1340:16: warning: 'input' defined but not used [-Wunused-function]
1340 |     static int input (void)
      |                  ^~~~~
lex.yy.c:1297:17: warning: 'yyunput' defined but not used [-Wunused-function]
1297 |     static void yyunput (int c, char * yy_bp )
      |                  ^~~~~~
gcc -W -Wall -c y.tab.c
gcc -W -Wall main.o util.o lex.yy.o y.tab.o -o cminus_parser -lfl
```

2.3. Execution Process

1. `./cminus_cimpl <testfile>.txt > <output>.txt` : Saves our result in an output file
2. `./cminus_cimpl <testfile>.txt`: Prints our result in the terminal
3. `diff -y <result>.txt <output>.txt`: Checks the differences between our output and the results

```
root@9282dad7c16e:/work# ./cminus_parser test.1.txt

C-MINUS COMPILATION: test.1.txt

Syntax tree:
  Function Declaration: name = gcd, return type = int
    Parameter: name = u, type = int
    Parameter: name = v, type = int
    Compound Statement:
      If-Else Statement:
        Op: ==
        Variable: name = v
        Const: 0
      Return Statement:
        Variable: name = u
      Return Statement:
```

```
root@9282dad7c16e:/work# diff result.1.txt my_result.1.txt
2c2
< C-MINUS COMPILATION: ./test.1.txt
---
> C-MINUS COMPILATION: test.1.txt
root@9282dad7c16e:/work#
```

3. Implementation Details

The parser is composed of several key files that work together to build the AST.

3.1. main.c

This file is the entry point. The preprocessor macros were modified as indicated in the guide to create a parser-only build that prints the tree:

- `#define NO_PARSE FALSE`
- `#define NO_ANALYZE TRUE`
- `#define TraceParse TRUE`

3.2. globals.h (Data Structure)

This file defines the fundamental data structures for the AST.

- **NodeKind**: Expanded to include DeclK (declaration nodes), in addition to StmtK and ExpK.
- **StmtKind, ExpKind, DeclKind**: New enum types were created to represent all nodes in the C-Minus grammar (e.g., FunK, VarK, CompoundK, WhileK, CallK, etc.).
- **TreeNode**: This is the main AST structure. It was modified to:
 - Include DeclKind decl in the kind union.
 - Add an int isArray; field to handle array declaration syntax (e.g., int x[5]).

3.3. util.c (Utility Functions)

This file contains the functions for creating and printing the AST.

- **newDeclNode():** This new "constructor" function was added to create nodes of type DeclK.
- **printTree():** This function was completely rewritten to handle the new node types (DeclK, StmtK, ExpK). It traverses the tree (children and siblings) and prints the AST with the correct indentation, matching the output format in the PDF.
- **printType():** A new helper function created for printTree, responsible for printing complex data types like int or void[].

3.4. cminus.y (The Grammar)

This is the heart of the parser.

- **Token Definitions:** All C-Minus tokens (IF, ELSE, WHILE, RETURN, INT, VOID, NE, LE, GE, LBRACE, RBRACE, etc.) were declared using %token.
- **Operator Precedence:** Precedence and associativity rules were defined (e.g., %left PLUS MINUS, %right ASSIGN) so the parser correctly resolves ambiguity in mathematical expressions (e.g., $3 + 4 * 5$).
- **Grammar Rules and Actions:** All rules from the C-Minus grammar were implemented. Each rule has a **semantic action** (C code in {...}) that:
 1. Creates a new TreeNode (using newStmtNode, newExpNode, newDeclNode).
 2. Links the child nodes (returned by sub-rules, e.g., \$1, \$3) to the new node's (\$\$) child[] array.
 3. Manages lists (like declaration_list) by linking nodes via the sibling pointer.

4. Complications and Solutions

Several critical problems arose during development, requiring debugging and redesign.

4.1. Problem: Token Definition Conflict

- **Complication:** The compilation failed with dozens of "redeclaration" errors for tokens like IF, ELSE, ID, NUM, etc.
- **Cause:** globals.h was defining the tokens in an enum TokenType, while cminus.y was defining them using %token. This caused the Yacc-generated y.tab.h and globals.h to conflict.
- **Solution:** The token definitions were centralized. All tokens were removed from the enum in globals.h (leaving only ENDFILE). cminus.y was made the **single source of truth**. All other files that needed token definitions (like util.c and cminus.l) were modified to #include "y.tab.h".

4.2. Problem: Incorrect AST Output (tokenString Corruption)

- **Complication:** Initial tests failed. The printed AST showed incorrect names and values (e.g., Function Declaration: name = }, ... instead of name = gcd, ...).
- **Cause:** Semantic actions in `cminus.y` (e.g., `fun_declaration`) do not execute until the entire rule has been seen. By then, the global `tokenString` variable (used by Flex) had already been overwritten by subsequent tokens (like `{` or `}`). The action was copying the *current* value of `tokenString`, not the value it had when the ID was found.
- **Solution:** Following the hint in the PDF, **intermediate non-terminal rules** were implemented:

```
/* Rule to immediately capture the value of an ID */
identifier : ID
{
    $$ = newExpNode(IdK); /* Create a temporary node */
    $$->attr.name = copyString(tokenString);
    $$->lineno = lineno;
}
;
```

- These rules reduce *immediately* after the scanner sees an ID or NUM, allowing the value of `tokenString` to be captured and copied into a temporary node. The higher-level rules (like `fun_declaration`) were then modified to use this node (`identifier` instead of `ID`) to get the correct name.

4.3. Problem: shift/reduce Conflict and PDF Penalty

- **Complication:** The compilation, while functional, showed warning: 1 shift/reduce conflict. The PDF guide explicitly states that *all* Yacc conflicts must be removed or be penalized.
- **Cause:** The grammar for `selection_stmt` (the if-then-else) was ambiguous, creating the classic "dangling else" problem. Using `%nonassoc ELSE` resolved the ambiguity, but **did not remove the warning**.
- **Solution:** The statement grammar was rewritten to be **explicitly unambiguous**. The `%nonassoc ELSE` rule was removed, and the rules for `statement` and `iteration_stmt` were replaced with a set that distinguishes between "matched" and "unmatched" statements:
 - **matched_stmt:** Statements that cannot cause ambiguity (e.g., if-else, a while with a "matched" body, `compound_stmt`, `return_stmt`).
 - **unmatched_stmt:** Statements that can (e.g., if without else, a while with an "unmatched" body).

This new grammar resolved the ambiguity at the syntax level, completely eliminating the shift/reduce warning and producing a clean compilation.

5. Results and Verification

The final parser was validated using the test files test.1.txt and test.2.txt. The parser's output was redirected to files (my_result.1.txt and my_result.2.txt) and compared to the expected results (result.1.txt and result.2.txt) using diff.

As the screenshot shows, the diff utility reported no differences in the AST, confirming the implementation is correct. The only difference was the input file path (test.2.txt vs. ./test.2.txt), which is cosmetic and does not affect the tree.

```
root@9282dad7c16e:/work# ./cminus_parser test.1.txt > my_result.1.txt
root@9282dad7c16e:/work# diff result.1.txt my_result.1.txt
2c2
< C-MINUS COMPILATION: ./test.1.txt
---
> C-MINUS COMPILATION: test.1.txt
root@9282dad7c16e:/work# ./cminus_parser test.2.txt > my_result.2.txt
root@9282dad7c16e:/work# diff result.2.txt my_result.2.txt
2c2
< C-MINUS COMPILATION: ./test.2.txt
---
> C-MINUS COMPILATION: test.2.txt
root@9282dad7c16e:/work#
```

```
C-MINUS COMPILATION: test.1.txt

Syntax tree:
Function Declaration: name = gcd, return type = int
  Parameter: name = u, type = int
  Parameter: name = v, type = int
  Compound Statement:
    If-Else Statement:
      Op: ==
      Variable: name = v
      Const: 0
    Return Statement:
      Variable: name = u
    Return Statement:
      Call: function name = gcd
      Variable: name = v
      Op: -
      Variable: name = u
      Op: *
      Op: /
      Variable: name = u
      Variable: name = v
      Variable: name = v
Function Declaration: name = main, return type = void
  Void Parameter
  Compound Statement:
    Variable Declaration: name = x, type = int
    Variable Declaration: name = y, type = int
    Assign:
      Variable: name = x
      Call: function name = input
    Assign:
      Variable: name = y
      Call: function name = input
    Call: function name = output
    Call: function name = gcd
      Variable: name = x
      Variable: name = y
```

6. Conclusion

The syntax analyzer for C-Minus was successfully implemented. The program generates a correct AST and produces no Yacc conflicts, meeting all project requirements. The primary challenges were managing token value capturing (a timing issue) and resolving grammar ambiguity at the rule level to achieve a clean compilation.